

TTFL: Trust-Flow-Driven Trusted Federated Learning for Edge Computing

Jiakai Zhou, Chao Bu, Shuo Zhang, Xinyu Liu
School of Computer Science and Engineering
Tianjin University of Technology, Tianjin, China
Email: wllsp0@stud.tjut.edu.cn, bc_0722@163.com

Abstract—Federated learning (FL) must assess both the execution that produced an update and the update’s effect on the global model. This is difficult under Non-IID data: a useful long-tail client can look anomalous, whereas an attacker can build a favorable history before submitting a harmful update. We present Trust Flow, which separates hardware-anchored admission, current content evidence, long-term utility, and short-term risk before layer-wise aggregation. A Kalman trust filter handles attestation and runtime uncertainty; a decayed Beta stream retains contribution history; normalized risk probes drive a fast EMA and persistent state transitions. Layer thresholds, clipping, and data-size-weighted survivor re-normalization then control each update’s influence. Experiments use Flower/PyTorch, CIFAR-10 adapted ResNet-18, two Intel/AMD TEE-capable platforms, five seeds per platform (ten platform-seed runs), and six concurrent attack classes. Targeted ASR is reported separately for trigger, clean-label, and semantic attacks and then macro-averaged. Under the stated 30% malicious protocol, Trust Flow reaches $92.31 \pm 0.09\%$ clean accuracy and $10.21 \pm 0.05\%$ macro target ASR, with no permanent false ban in the tested runs.

Index Terms—Federated learning, edge computing, trusted execution environment, risk modeling, layered aggregation, backdoor defense

1. Introduction

FL enables edge devices to train a shared model through parameter exchange while keeping their raw samples local [1], [2]. In an open edge deployment, however, the server must assess both the execution that produced an update and its effect on the global model. This assessment becomes difficult under Non-IID data: a legitimate long-tail client may repeatedly deviate from a reference direction, whereas an attacker may build a favorable record before submitting a harmful update. A single reputation score makes these cases difficult to separate. Tightening its threshold can remove useful heterogeneous contributions, while allowing past performance to compensate for current anomalies can delay the response to an attack. The resulting challenge is to preserve evidence of long-term contribution without allowing that evidence to conceal short-term risk.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3], [4],

[5], [6], [7]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and collusion [8], [9], [10], [11]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [12], [13], [14], [15]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy addresses this tension by maintaining historical utility and short-term risk as separate states. Historical utility governs how a client retains or recovers its contribution, while current risk can restrict an update despite a favorable earlier record. These states connect execution admission, content auditing, and layer-wise aggregation: Intel TDX establishes the execution boundary, current evidence determines the submitted update’s influence, and persistence rules govern subsequent participation. Assigning evidence to these different decisions allows the framework to distinguish a temporarily unhelpful client from one that presents sustained risk. The contributions are:

- 1) A dual-stream state representation gives historical contribution and short-term risk separate roles, preserving benign-client retention while allowing a rapid response to attacks.
- 2) A state-to-aggregation policy links recoverable suspicion, persistent-risk escalation, and layer-specific weighting. Content quality can lower a contribution without, by itself, forcing irreversible exclusion.
- 3) An empirical analysis relates the single-stream and HistPerf-only ablations to the retention–detection tradeoff, and separates the effects of risk probes from those of weight re-normalization.

2. Background and Related Work

Federated learning in edge computing. Local training and server aggregation allow clients to collaborate without transferring raw samples, but leave the server dependent on the evidence accompanying each update. Heterogeneous data distributions complicate that assessment because a useful long-tail contribution may differ from the majority. Execution provenance and update quality consequently serve different purposes: the former establishes how an update

was produced, while the latter determines how it should contribute to learning.

Poisoning and backdoor attacks. Sign flipping and gradient scaling disrupt optimization by changing update directions or magnitudes. Trigger, clean-label, and semantic backdoors instead seek targeted errors while preserving plausible main-task behavior, so their updates can remain close to an expected direction [16], [17]. Across rounds, an attacker may also accumulate a favorable history before injecting a harmful update, making current behavior and past contribution potentially conflicting sources of evidence.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3], [4], FLTrust weights updates against a clean direction [5], and FoolsGold and FLDetector use cross-client or cross-round evidence [6], [7]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [8], [9], [10], [11], [18], [19], [20]. TMT-FL and related systems connect execution integrity to model training [15]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The experiments use classical methods as matched aggregation baselines and stateful variants to isolate this temporal interface.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [12], [13], [14], [15], [21]. A quote authenticates the measured path, freshness, and update provenance. TTFL combines that hardware evidence with privacy-aware verification and cloud-side semantic auditing before assigning aggregation privileges.

3. System Model and Threat Assumptions

Figure 1 presents the client-edge-cloud environment, consisting of a central aggregation server S and K edge clients $\mathcal{C} = \{c_1, \dots, c_K\}$. In round t , admitted clients train from the distributed model $W^{(t-1)}$ and submit updates $\Delta W_k^{(t)}$ for auditing and aggregation. The server then computes $W^{(t)} = W^{(t-1)} + \text{Agg}(\{\Delta W_k^{(t)}\})$, with the global learning objective

$$\min_W \mathcal{L}(W) = \sum_{k=1}^K p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \quad (1)$$

The client side performs local training and supplies an attestation-bound report, while privacy-aware verification exposes the declared data and runtime statistics needed for admission and audit. Raw samples remain local. In the evaluated deployment, CPU-side TEE/TDX protects the measured admission agent, device keys, report integrity,

nonce freshness, and admission state; GPU training memory and the semantic correctness of labels, samples, objectives, and updates are outside that boundary. Thus an attacker may produce a signed poisoned update inside an otherwise measured training process, while post-attestation host tampering is excluded. Using the submitted update and this evidence, the cloud server evolves the client’s trust states and determines its aggregation privilege. Because these decisions require individual updates and audit evidence, cryptographic secure aggregation is not enabled in the evaluated protocol.

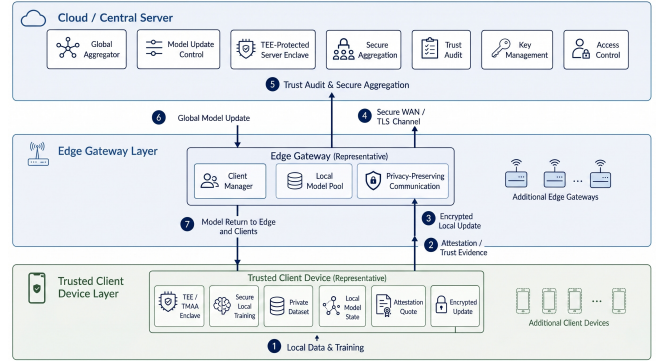


Figure 1. TEE-anchored client-edge-cloud architecture.

Threat model. Figure 2 summarizes the attack surface. The server is honest-but-curious and follows the aggregation protocol. It can see individual updates and audit statistics, but it does not change the stated state-transition rules. An admitted client may alter labels, samples, or optimization objectives and may conduct label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, or gradient scaling.

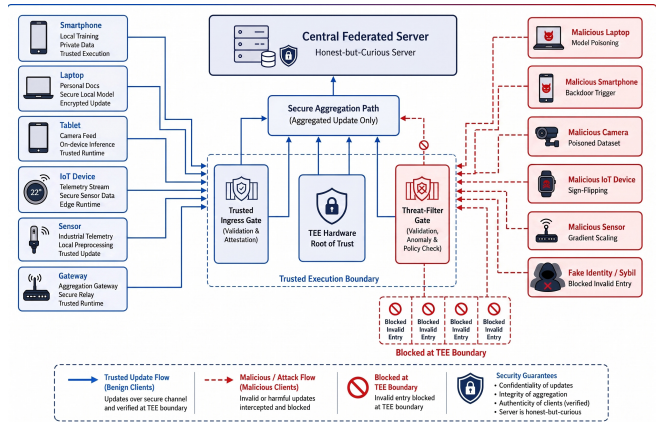


Figure 2. Composite poisoning and penetration threat model.

The client execution path is instantiated on Intel- and AMD-capable TEE platforms. The secure-aggregation path drawn as an optional deployment component in Fig. 2 is disabled in the evaluated protocol because the server must inspect individual evidence. The CPU-side TEE protects attestation-critical code, device keys, report integrity, nonce

freshness, and the binding among client identity, round number, global-model digest, update digest, and signing key. It does not authenticate labels, samples, optimization objectives, semantic intent, or GPU memory. Consequently, the six attacks below may operate inside a valid measured training process; tampering with the update after attestation or attacks on the TEE implementation itself are excluded. Physical probing and advanced microarchitectural side channels are also excluded. The assumed malicious fraction is below 50%; the 50% experiment is a boundary stress test rather than evidence of a Byzantine guarantee. The main experiments use a 500-sample server proxy for reference and probes. The peer-reference branch only supplies a direction when that clean direction is unavailable; clean-loss and activation probes still require the proxy and are not silently treated as reference-free.

Design goals. Trust Flow is designed to maintain convergence under the stated attacks, avoid permanent false bans of long-tail clients, and keep the assurance boundary explicit. Execution, current content, historical utility, and recent risk retain separate meanings until layer-wise aggregation. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. The auxiliary security path measures approximately 11 ms per admitted-client round, while device-side TEE Quote generation, signing, and lightweight monitoring measure approximately 12–15 ms per admitted-client round. These component measurements are reported separately from server processing and end-to-end communication.

4. Trust-Flow Framework

Each FL round passes through four linked decision stages: admission and runtime assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Global model update and feedback close the five-phase loop shown in Fig. 3.

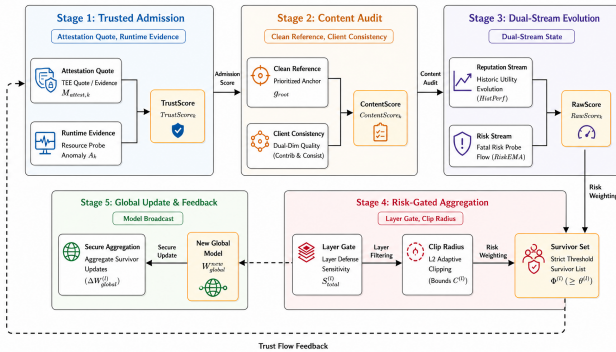


Figure 3. Overview of the five-phase trust-flow defense loop.

The stages in Fig. 3 pass progressively different evidence to the aggregation decision. Admission and runtime assessment produce *TrustScore*, content auditing produces *ContentScore*, and state evolution accumulates evidence in *HistPerf* and *RiskEMA*. Their combination yields

RawScore, which controls layer privileges. The server first removes clients that fail the admission proof, then scores the remaining updates against the reference and peer group. Keeping contribution and risk distinct allows a weak single-round contribution to reduce influence without immediately ending a benign client’s opportunity to recover.

4.1. Admission and Content Auditing

Each admitted client passes a TEE admission check. Let $M_{\text{attest},k} \in \{0, 1\}$ indicate whether the quote, nonce freshness, code identity, round identifier, current global-model digest, update digest, and signing-key binding are valid. During initialization, the protected management agent records the standardized behavior sequence $\mathcal{X}_k^{(t)} = \{\Delta\text{grad}, \Delta\text{loss}, \text{instr_dist}\}$. The concatenated histogram entropy is normalized as $h_k^{(t)} = H(\mathcal{X}_k^{(t)})/H_{\max} \in [0, 1]$. The observation and measurement-noise terms are

$$R_k^{(t)} = \exp(\lambda h_k^{(t)}),$$

$$Z_k^{(t)} = \text{clip}\left(w_a M_{\text{attest},k} + (1 - w_a)(1 - h_k^{(t)}), 0, 1\right). \quad (2)$$

where $w_a = 0.7$ and $\lambda = 1.0$. Entropy measures uncertainty; it is not a maliciousness label. The scalar random-walk Kalman recursion is

$$T_k^{-(t)} = T_k^{-(t-1)}, \quad P_k^{-(t)} = P_k^{-(t-1)} + Q, \quad (3)$$

$$K_k^{(t)} = \frac{P_k^{-(t)}}{P_k^{-(t)} + R_k^{(t)}},$$

$$T_k^{(t)} = T_k^{-(t)} + K_k^{(t)} \left(Z_k^{(t)} - T_k^{-(t)} \right), \quad (4)$$

$$P_k^{(t)} = (1 - K_k^{(t)}) P_k^{-(t)}.$$

with $Q = 0.01$, $T_k^{(0)} = 1$, and $P_k^{(0)} = 0.25$. The admission score is

$$\text{TrustScore}_k^{(t)} = M_{\text{attest},k} \text{clip}\left(\frac{T_k^{(t)}}{1 + P_k^{(t)}}, 0, 1\right). \quad (5)$$

A newly attested client starts in **NORMAL**; first-round evidence may reduce its current aggregation privilege but cannot directly create an irreversible blacklist.

The server next constructs a reference direction. In the reported protocol, $g_{\text{root}} = g_{\text{root_clean}}$ is obtained from a server-held balanced proxy set of 500 CIFAR-10 test/reference samples (50 per class), disjoint from client training data and final evaluation. If a clean direction is unavailable, the fallback is formed from admitted updates with $\omega_k^{\text{ref}} = \text{TrustScore}_k(1 - \text{RiskEMA}_k^{\text{prev}})^2$. This fallback changes only the reference direction; the clean-loss and

activation probes still require proxy inputs. For each update, content quality combines reference and peer consistency:

$$S_{\text{contrib},k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{\text{root}})), \quad (6)$$

$$\rho_{kj} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j)), \quad (7)$$

$$S_{\text{consist},k} = \frac{\sum_{j \neq k} \text{TrustScore}_j \rho_{kj}}{\sum_{j \neq k} \text{TrustScore}_j + \varepsilon}, \quad (8)$$

$$\text{ContentScore}_k = \frac{(1 + \beta_{\text{fusion}}^2) S_{\text{consist},k} S_{\text{contrib},k}}{D_k}, \quad (9)$$

$$D_k = \beta_{\text{fusion}}^2 S_{\text{consist},k} + S_{\text{contrib},k} + \varepsilon. \quad (10)$$

Here $\alpha_1 = 0.6$, $\alpha_2 = 0.4$, and $\beta_{\text{fusion}} = 2.0$. The score describes current update quality; it is not itself a permanent reputation decision.

4.2. Dual-Stream State Evolution

Cold start and first-round evidence. At $t = 1$, $\text{HistPerf}_k^{(0)}$ and $\text{RiskEMA}_k^{(0)}$ represent initial conditions rather than accumulated client behavior; the runtime trust estimate and covariance likewise start from their filter initialization. In the reference-assisted experimental setting, the server can construct $g_{\text{root},\text{clean}}$ without a client history. Once the first local updates arrive, runtime reports, reference alignment, peer consistency, and the available risk probes provide current-round evidence. These checks do not require a previously established reputation: they assess the first submitted update, while the two temporal streams begin accumulating evidence for later decisions.

A low content score can reflect either a useful client's skewed data or a harmful update. Since those cases require different participation decisions, Trust Flow retains historical utility and short-term risk separately. Let $q_k^{(t)} \in [0, 1]$ denote normalized content quality. With the coefficients in Table 2 and nonnegative trust weights, $\text{ContentScore}_k^{(t)}$ already lies in this interval and serves directly as $q_k^{(t)}$. The unit-evidence form of the decayed Beta model adds $q_k^{(t)}$ to positive evidence and $1 - q_k^{(t)}$ to negative evidence:

$$\begin{aligned} \alpha_k^{(t)} &= \beta_h \alpha_k^{(t-1)} + q_k^{(t)}, \\ \beta_k^{(t)} &= \beta_h \beta_k^{(t-1)} + 1 - q_k^{(t)}. \end{aligned} \quad (11)$$

Here $0 < \beta_h < 1$ is the history retention factor, distinct from the negative-evidence evidence mass, and the reported protocol initializes $\alpha_k^{(0)} = \beta_k^{(0)} = 1$, so $\text{HistPerf}_k^{(0)} = 0.5$. The historical utility is

$$\text{HistPerf}_k^{(t)} = \frac{\alpha_k^{(t)}}{\alpha_k^{(t)} + \beta_k^{(t)} + \varepsilon}. \quad (12)$$

Decay allows later observations to revise the contribution history rather than preserving early evidence indefinitely.

The history channel changes slowly so that a few weak contributions do not erase a client's accumulated utility.

For a legitimate long-tail client, low similarity can lower aggregation privilege while leaving later rounds available for recovery. The risk channel instead retains recent warnings through $\text{RiskEMA}_k^{(t)} = \beta_r \text{RiskEMA}_k^{(t-1)} + (1 - \beta_r) \text{Risk}_k^{\text{inst}}$, where $\text{Risk}_k^{\text{inst}}$ is the maximum current probe. Taking the maximum preserves a strong warning across probe channels; maintaining risk separately prevents a favorable history from altering that risk estimate. The state-transition rules then determine whether the evidence warrants review, quarantine, or permanent exclusion.

The separation changes how borderline clients are treated. A client with weak content quality but no rising risk can remain in the system with reduced influence, giving later rounds an opportunity to clarify whether the deviation was benign. A client with favorable historical utility but rising attack risk instead faces restricted aggregation privileges. Low historical utility can lead to temporary soft isolation, while the risk stream distinguishes sustained malicious behavior from a recoverable contribution deficit.

The two streams meet only when computing aggregation privilege:

$$\begin{aligned} \text{RawScore}_k^{(t)} &= (\text{TrustScore}_k^{(t)})^\alpha \\ &\quad \times (\text{ContentScore}_k^{(t)})^\beta (\text{HistPerf}_k^{(t)})^\gamma \\ &\quad \times (1 - \text{RiskEMA}_k^{(t)})^p. \end{aligned} \quad (13)$$

The resulting state machine distinguishes NORMAL, SUSPECT, QUARANTINE, and BLACKLIST, as shown in Fig. 4. A sleeper-burst attacker may accumulate historical utility during benign rounds, but a later trigger injection activates the risk probes and restricts its influence. A benign long-tail client follows a different path: low current similarity mainly affects its utility stream, and later positive contributions allow recovery. Keeping those paths separate avoids using one threshold for both contribution filtering and permanent exclusion.

The current-round states enter the reported aggregation decision. Evidence is computed first, the state transition is applied second, the candidate set is formed third, and layer filtering and aggregation are performed last. A newly attested client can therefore lose current-round influence without being permanently banned; BLACKLIST becomes a gate on subsequent rounds.

TABLE 1. STATE-TRANSITION ORDER USED IN EVERY COMMUNICATION ROUND.

From	Trigger and persistence	Action
NORMAL	$u > \tau_h$ or $r \geq \tau_s$ for 2 submitted rounds	SUSPECT; current update remains conditional
SUSPECT	same condition for 3 rounds, or $\text{Risk}_k^{\text{inst}} \geq \tau_q$	QUARANTINE; current update excluded
QUARANTINE	$r \geq \tau_b$ for 4 rounds	BLACKLIST; next-round gate
S or Q	$u \leq \tau_h$ and $r < \tau_s$ for 2 rounds	recover one state level

Counters advance only on rounds with a submitted update and reset when the condition is not met. The severe

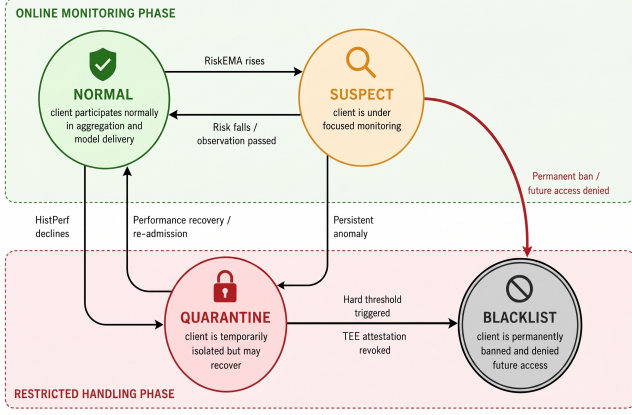


Figure 4. Client state-transition automaton driven by historical utility and short-term risk.

current-round quarantine rule has priority over recovery; a newly created BLACKLIST is applied as a gate to the next round, whereas QUARANTINE excludes the current update. This ordering is the one used for the reported metrics.

4.3. Risk-Gated Layered Aggregation

Backdoor effects may concentrate in deeper layers even when shallow features retain useful heterogeneous information. A uniform clipping rule then faces a tradeoff between suppressing the affected layers and preserving contributions elsewhere. Figure 5 shows how Trust Flow addresses this through layer-specific sensitivity, entry thresholds, and clipping radii. Each layer l combines privacy exposure, reference-gradient utility, and security deviation into a sensitivity score. The privacy term decreases with depth, the utility term reflects the relative reference-gradient norm of the layer, and the security term measures deviation from the reference direction. With the utility complement shown in Fig. 5, their combination is

$$S_{total}^{(l)} = w_p S_{priv}^{(l)} + w_u (1 - S_{util}^{(l)}) + w_s S_{sec}^{(l)}, \quad (14)$$

where w_p, w_u, w_s are the nonnegative component weights. The resulting sensitivity determines both the admission threshold and clipping radius:

$$\theta^{(l)} = \mu_{base} + \lambda_s S_{total}^{(l)}, \quad C^{(l)} = \frac{C_{base}}{S_{total}^{(l)} + \varepsilon_c}. \quad (15)$$

A layer showing stronger suspicious drift therefore has a higher entry threshold and a smaller clipping radius. Here $C_{base} > 0$ is the base update-norm scale, while $\varepsilon_c > 0$ stabilizes the denominator. Layers that remain close to the reference retain more capacity for benign heterogeneity.

Audit eligibility and aggregation eligibility. Let $\mathcal{D}_t$ contain the current valid submissions from clients that have not been permanently blacklisted. A quarantined client may supply evidence for recovery without contributing an update to the global model. After auditing these submissions,

let $s_k^{(t)}$ denote the resulting client state. The aggregation candidates are

$$\mathcal{A}_t = \{k \in \mathcal{D}_t \mid s_k^{(t)} \in \{\text{NORMAL}, \text{SUSPECT}\}\}. \quad (16)$$

The set $\mathcal{A}_t$ is denoted by $\mathcal{A}$ in Fig. 5. SUSPECT retains conditional eligibility under the score and layer gates, whereas QUARANTINE and BLACKLIST have no aggregation weight. State transitions are applied before forming $\mathcal{A}_t$: a quarantine decision removes the current update, and the current-round score is used for the remaining candidates. Recovery restores eligibility only after the state transition has occurred.

The round therefore proceeds from valid submissions to current evidence, state updates, candidate selection, and finally layer filtering and aggregation. The surviving clients are $\Phi_t^{(l)} = \{k \in \mathcal{A}_t \mid \text{RawScore}_k^{(t)} \geq \theta^{(l)}\}$, written as $\Phi^{(l)}$ below for brevity. For a nonempty survivor set, their weights are re-normalized:

$$\begin{aligned} \tilde{w}_k^{(l)} &= \frac{p_k \text{RawScore}_k^{(t)}}{\sum_{j \in \Phi^{(l)}} p_j \text{RawScore}_j^{(t)} + \varepsilon}, \\ \widehat{W}_k^{(l)} &= \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \end{aligned} \quad (17)$$

The final layer update is $\Delta W_{global}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{W}_k^{(l)}$. When $\Phi^{(l)}$ is empty, the client with the largest RawScore is retained with unit normalized weight and the same clipping radius; only an empty admitted set leaves the layer unchanged. The data-size factor p_k preserves the stated global objective after survivor filtering, and re-normalization prevents an implicit learning-rate decay.

Layer-wise control allows the same client update to receive different treatment across the network. When a semantic backdoor mainly alters deep features, sensitivity-dependent thresholds and clipping can constrain those layers while allowing more of a consistent shallow-layer update to contribute. Re-normalization addresses a separate optimization effect: removing clients also removes aggregation weight, which would otherwise reduce the update magnitude merely because filtering occurred. Projecting the surviving scores back to a valid aggregation simplex separates that weight loss from the intended clipping operation.

Interpretation of the history stream. The decayed Beta evidence has a finite effective mass and therefore behaves as a startup-corrected exponential smoother: early observations have larger gain, while later evidence is retained with factor β_h . This explains recovery of a benign long-tail client under the stated bounded-observation assumptions; the independent risk EMA preserves short-lived attack warnings instead of averaging them into historical utility.

4.4. Risk Probe Computation

All instantaneous risk channels are mapped to $[0, 1]$ before fusion. For a candidate update, the geometric and

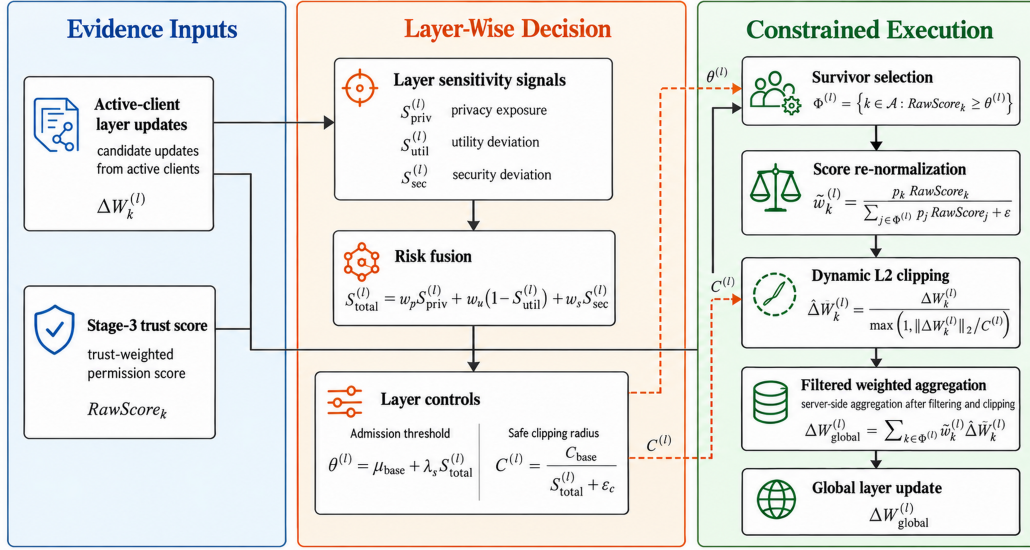


Figure 5. Risk-gated layer-wise clipping and differentiated aggregation.

amplitude probes are

$$r_{\text{grad},k}^{(t)} = \text{clip}\left(\frac{1 - \cos(\Delta W_k, g_{\text{root}})}{2}, 0, 1\right), \quad (18)$$

$$r_{\text{amp},k}^{(t)} = \text{clip}\left(\frac{\|\Delta W_k\|_2}{\text{median}_j \|\Delta W_j\|_2 + \varepsilon} - 1, 0, 1\right). \quad (19)$$

The clean-loss probe uses the server proxy:

$$r_{\text{probe},k}^{(t)} = \text{clip}\left(\frac{\mathcal{L}_{\text{proxy}}(W + \Delta W_k) - \mathcal{L}_{\text{proxy}}(W)}{\delta_{\text{clean}} + \varepsilon}, 0, 1\right), \quad (20)$$

where δ_{clean} is fixed from the median plus three MAD values on a trusted calibration set and is not tuned on the final test set. The activation-drift probe compares 32-bin, smoothed histograms from the last convolutional block and pre-classifier on the same proxy inputs:

$$r_{\text{trigger},k}^{(t)} = \text{clip}\left(\frac{\text{KL}(\mathcal{H}(A_{\text{base}}) \parallel \mathcal{H}(A_k))}{\delta_{\text{trigger}} + \varepsilon}, 0, 1\right). \quad (21)$$

It is an activation-drift probe; it does not assume that the server knows an attacker’s trigger. Peer disagreement is similarly normalized as $r_{\text{peer},k} = \text{clip}(1 - \cos_w(\Delta W_k, \Delta W_{\text{peer}}), 0, 1)$. The instantaneous risk is

$$\begin{aligned} \text{Risk}_k^{\text{inst}} &= \max\{r_{\text{grad},k}, r_{\text{amp},k}, r_{\text{probe},k}, \\ &\quad r_{\text{trigger},k}, r_{\text{peer},k}\}, \\ \text{RiskEMA}_k^{(t)} &= \beta_r \text{RiskEMA}_k^{(t-1)} + (1 - \beta_r) \text{Risk}_k^{\text{inst}}. \end{aligned} \quad (22)$$

with $\beta_r = 0.85$. The maximum preserves a strong warning while the common range prevents KL, loss, and norm scales from competing by units.

4.5. Assurance Composition and Reference Role

The evidence sources complement one another because each resolves a different uncertainty. Attestation and privacy-aware verification establish execution provenance and bounded statistics for admission, while cloud auditing examines the submitted update against reference and peer evidence. Temporal states retain the distinction between a transient deviation and persistent risk. Passing the measured execution path therefore leads into semantic auditing; it does not bypass it. Likewise, low similarity can reduce contribution without serving as sufficient evidence for permanent exclusion.

A clean validation gradient supplies a stable reference when available, while the peer-reference branch constructs a guiding direction from admitted updates when it is unavailable. Reference agreement contributes to content quality alongside peer consistency; historical contribution and recent risk then influence participation. The reference is therefore one input to auditing rather than a substitute for the dual-stream policy.

4.6. From Separate States to Participation Decisions

The distinction between historical contribution and recent risk matters most for clients near a participation boundary. A client with low utility but low risk may receive reduced influence or reversible review, allowing later evidence to clarify whether its deviation was benign. A client with substantial past utility and rising risk instead faces restrictions despite that history. Persistent high risk is required for irreversible blacklisting, so the same threshold need not serve simultaneously as a contribution filter and a permanent-exclusion rule.

For fixed admission factor, content score, and historical utility, write $B_k = (TrustScore_k)^\alpha (ContentScore_k)^\beta (HistPerf_k)^\gamma$. With $p > 0$ and $0 \leq r < 1$, the score satisfies

$$\frac{\partial[B_k(1-r)^p]}{\partial r} = -pB_k(1-r)^{p-1} \leq 0, \quad (23)$$

$$r = RiskEMA_k^{(t-1)}.$$

At $r = 1$, the score is zero. This statement concerns the unnormalized score with the other factors fixed; layer selection and normalization determine the update that is eventually applied. SUSPECT and QUARANTINE are recoverable states, allowing later evidence to revise a temporary restriction. Low similarity alone is therefore not interpreted as maliciousness.

The mechanism-level evidence is separated across the ablations. The reputation ablation compares single-stream variants, HistPerf-only, and the complete dual stream on false bans and detection. The component ablations isolate probe contributions and re-normalization. The first comparison tests the temporal policy; the second tests the evidence supplied to it and the optimization behavior after filtering.

5. Experiments

5.1. Setup

Table 2 summarizes the experimental configuration. Experiments use Flower 1.5.0 and PyTorch with a CIFAR-10-adapted ResNet-18: the first convolution is 3×3 with stride one, the initial max-pooling layer is removed, and the classifier has ten outputs. Training runs on two Intel/AMD TEE-capable CPU platforms with five NVIDIA A10 GPUs, Ubuntu 20.04.6 LTS, and CUDA 12.8. The CPU-side TEE handles proof, monitoring, report generation, and admission state; GPU training is outside the CPU TEE boundary. Each platform uses five independent seeds, so aggregate means and standard deviations use ten platform-seed runs. State trajectories use one fixed seed per platform.

TABLE 2. EXPERIMENTAL CONFIGURATION AND MAIN HYPERPARAMETERS.

Category	Setting	Value
FL task	CIFAR-10-adapted ResNet-18, admitted clients	$K = 20$
Optimization	Local epochs; lr; momentum; rounds	3; 0.001; 0.9; 30
Data	Shared pool; exclusive pool; client groups	5000; 45000; IID 0–9, $\alpha = 1.0$ 10–14, $\alpha = 0.1$ 15–19
Admission	$w_a, \lambda, Q; (T_0, P_0, RiskEMA_0)$	0.7, 1.0, 0.01; (1, 0.25, 0)
Fusion	$\alpha_1, \alpha_2, \beta_{fusion}; \beta_h, \beta_r$	0.6, 0.4, 2.0; 0.9, 0.85
States	Thresholds; persistence	0.26/0.64/0.80/0.90; 2/3/4
Layers	$(\alpha, \beta, \gamma, p); (\mu_{base}, \lambda_s)$	(3.0, 1.0, 0.5, 1.0); (0.4, 1.2)
Layers	(τ_p, w_p, w_u, w_s)	(3.0, 0.3, 0.3, 0.4)

The 50,000 CIFAR-10 training images are divided into a shared 5,000-sample client pool and 45,000 exclusive

samples. The shared pool is replicated across clients; the exclusive samples use IID clients 0–9, Dirichlet $\alpha = 1.0$ clients 10–14, and Dirichlet $\alpha = 0.1$ clients 15–19. The server holds a separate balanced 500-sample proxy (50 per class) that is disjoint from client training and final evaluation. It is supplied to reference-dependent methods; FedAvg, Krum, and Trimmed Mean do not use it.

The formal 30% setting uses six malicious clients among 20 admitted clients. C0 performs untargeted label flipping with $\gamma = 0.5$; C1 injects a fixed lower-right 3×3 trigger into 20% of local samples and targets class 0; C2 performs clean-label poisoning on 20% of local samples with the same target; C3 applies a semantic attribute bias to 20% of local samples and targets class 0; C4 performs untargeted sign flipping; and C5 scales its update by 100. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload exercise Phase-One admission and are intercepted before aggregation. C1–C3 are targeted attacks. C0, C4, and C5 are evaluated using clean accuracy, loss, convergence, and state transitions rather than ASR. The attacker is allowed to produce a signed poisoned update inside the measured training path; post-attestation host tampering is outside the threat model.

For each targeted attack $a \in \{\text{trigger, clean, semantic}\}$, let $\mathcal{B}_a$ be its attack-specific test subset and y_a^* its target. We report

$$ASR_a = \frac{|\{x \in \mathcal{B}_a : \arg \max f(x) = y_a^*\}|}{|\mathcal{B}_a|}, \quad (24)$$

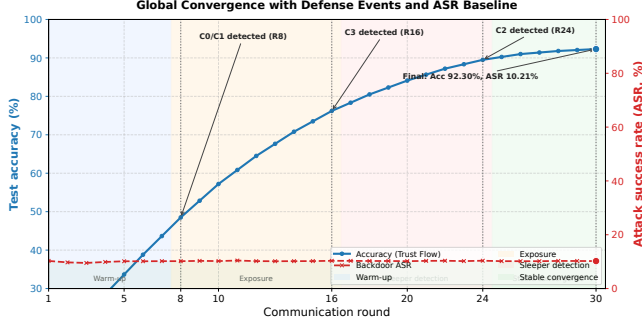
$$ASR_{\text{target}} = \frac{1}{3} \sum_{a \in \{\text{trigger, clean, semantic}\}} ASR_a. \quad (25)$$

The three attack-specific traces are retained; the headline target ASR is their macro average and is not a mixture with untargeted attacks. Accuracy, loss, per-round review, final permanent-ban FPR, participation, and recovery are reported with their own denominators. Means and standard deviations use the ten platform-seed runs; client-state trajectories use one fixed seed per platform. Each run has 30 communication rounds and three local epochs.

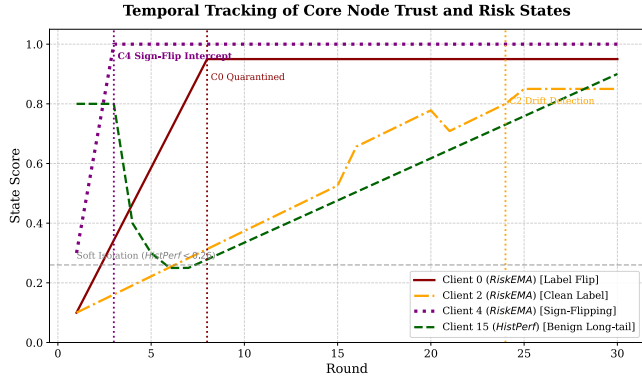
5.2. Overall Defense Performance

Figure 6 collects convergence, attack-specific ASR, and client-state trajectories. Under the 30% mixed-attack setting, Trust Flow reaches 92.31% accuracy and 10.21% ASR. Explicit poisoning and parameter attacks are detected early, while stealthier clean-label drift requires persistent probe evidence. A legitimate long-tail client may enter SUSPECT or QUARANTINE temporarily without accumulating the sustained risk associated with permanent exclusion. The trajectories connect this distinction to the aggregate accuracy and attack-suppression results.

The trajectories in Fig. 6(b) connect aggregate results to client-level decisions. Elevated risk is retained for label, trigger, semantic, sign, and scaling attacks, whereas a benign long-tail client’s weak contribution can cause a reversible



(a) Accuracy and the three targeted ASR traces.



(b) Node trust, risk, and state trajectories.

Figure 6. Main defense behavior under the original mixed-attack protocol.

review without a permanent ban. Later positive contributions allow its historical utility and participation to recover. Across the reported runs, benign clients have 100% final retention; the representative trajectories show how reversible review is exited. We do not use t-SNE as quantitative evidence; accuracy, attack-specific ASR, state transitions, actual participation, and recovery are the reported measures.

In the matched classical comparison in Table 3, Krum reaches 64.20% accuracy, Trimmed Mean 71.35%, and FLTrust 81.25%, while Trust Flow reaches 92.31%. These values do not establish a universal ranking. DeepSight, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA require attack-specific or privacy-preserving interfaces that are not reproduced here, so their absence is a comparison limitation. The 50% result remains an out-of-assumption stress test rather than evidence of a theoretical Byzantine boundary.

The classical rows use the same client partition, optimizer, attack proportion, communication budget, and proxy-access rule where applicable. The table is a protocol-specific comparison, not a claim of universal ranking; recent methods whose exact implementations were not re-run are intentionally left without numeric values.

5.3. Ablation, Sensitivity, and Overhead

Table 4 examines whether one state can serve both retention and detection. The aggressive single-stream variant

attains high recall at the cost of more permanent false positives, while the conservative variant reduces false positives but misses more malicious clients. HistPerf-only further illustrates the limitation of relying on contribution history: it retains benign clients but provides little attack sensitivity. The dual-stream result supports assigning those roles to separate states. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. Optional cryptographic secure aggregation remains outside the stated system boundary. The component and heterogeneity results are shown in Figs. 7 and 8.

The server timing covers aggregation and content audit. The measured auxiliary security-path overhead is approximately 11 ms per admitted-client round. Device-side TEE Quote generation, signing, and lightweight monitoring are measured separately at approximately 12–15 ms per admitted-client round. These values describe their respective measurement scopes rather than one end-to-end deployment latency. CPU-side TDX isolation does not by itself establish protection of GPU execution or GPU memory.

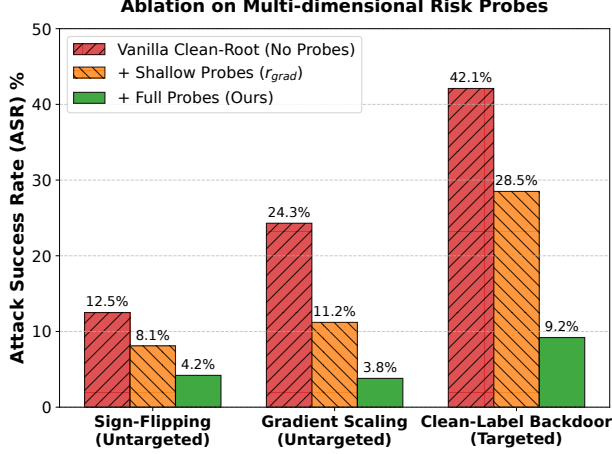
The first three FPR/TPR pairs in Table 4 are client-round review metrics from the original ablation logs; they are not final permanent-ban rates and therefore need not have the discrete denominator of a 14-benign/6-malicious final-state experiment. The complete Trust Flow row uses the final-ban definition over the ten platform-seed runs. Counting SUSPECT as recoverable review gives 88.33% per-round malicious-client coverage and a 14.79% benign-client review rate, while the final permanent-ban FPR is 0.00%. We report these quantities separately from actual participation and recovery: a benign client may be reviewed or temporarily excluded, but it is eligible to submit recovery evidence; the reported final retention is 100%, while the 14.79% figure is a reversible review rate rather than a participation rate. Figure 7 separates the effects of probe coverage and re-normalization: clean-reference and shallow checks handle explicit parameter attacks, while activation and clean-loss probes reduce the reported clean-label ASR to 9.2%. Survivor re-normalization avoids an implicit learning-rate decay after filtering. Figure 8 shows that Trust Flow remains near 92% accuracy at $\alpha = 0.1$.

The Dirichlet sweep in Fig. 8 holds label space, model, optimizer, and attack protocol fixed while varying data heterogeneity. As α decreases, benign updates become less aligned, making the filtering decisions of Krum and FLTrust more likely to reject legitimate drift. Trust Flow is less sensitive in the reported sweep because current content quality is considered together with historical utility and recent risk. The default $\beta_{fusion} = 2.0$ gives the best observed accuracy-ASR balance. The reported 2.10 s to 4.85 s runtime covers server-side aggregation and auditing, not end-to-end TDX deployment latency.

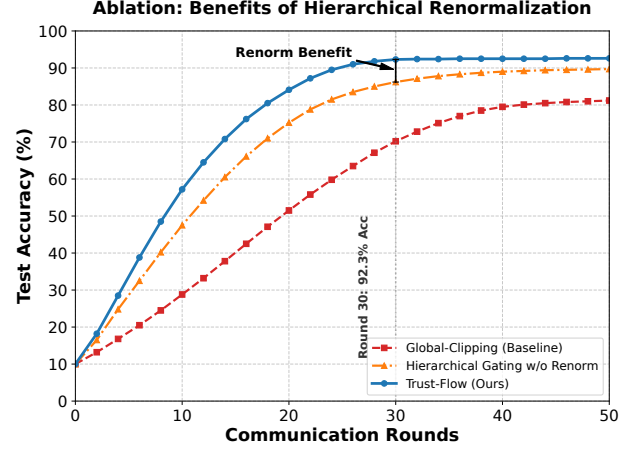
TABLE 3. PERFORMANCE COMPARISON UNDER 30% MIXED ATTACKS AND BOUNDARY STRESS TESTING.

Method/Setting	Core mechanism	Acc.	ASR	Perm. FPR [†]
FedAvg [2]	no defense	86.15 $\pm$ 1.20	92.34 $\pm$ 5.10	N/A
Krum [3]	Euclidean selection	64.20 $\pm$ 2.80	12.15 $\pm$ 0.90	N/A
Trimmed Mean [4]	coordinate trimming	71.35 $\pm$ 2.40	38.60 $\pm$ 4.20	N/A
FLTrust [5]	clean anchor weighting	81.25 $\pm$ 1.50	15.42 $\pm$ 1.10	N/A
Trust Flow	dual stream + layered gating	92.31 $\pm$ 0.09	10.21 $\pm$ 0.05	0.00
Boundary stress: 50% malicious	10/20 malicious	91.81 $\pm$ 0.10	10.46 $\pm$ 0.06	0.00

[†] Permanent-ban FPR is a client-level final-state metric and is defined only for the stateful Trust Flow protocol. Stateless aggregation baselines are marked N/A; reversible review and actual participation are reported separately. FLAME, FLDetector, and FreqFed are discussed as related methods but are not assigned numeric entries because their exact implementations and attack-specific interfaces were not re-run under this protocol.



(a) Multi-dimensional risk-probe ablation.



(b) Weight re-normalization effect.

Figure 7. Component-level ablation of risk probes and re-normalization.

TABLE 4. ABLATION AND OVERHEAD SUMMARY.

Study	Variant / architecture	Review FPR or overhead	Review TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	Dual-stream Trust Flow	0.00% FPR	100.00% TPR
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	~15 KB TrustReport	4.85 s incl. audit

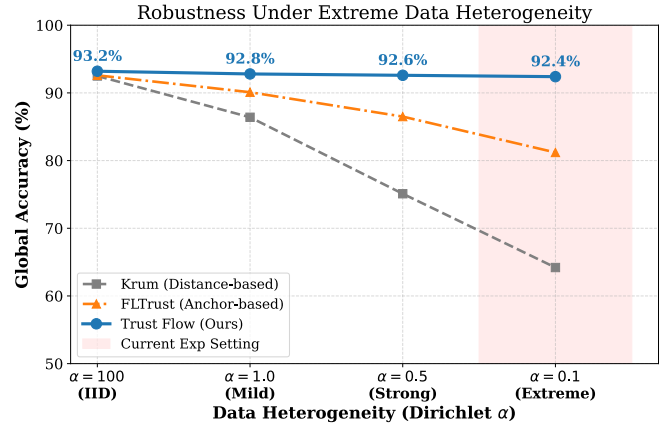


Figure 8. Robustness stress test under varying data heterogeneity.

6. Discussion and Conclusion

Trust Flow connects execution admission, privacy-aware statistical checks, content auditing, and layer-wise aggregation through separate historical-utility and short-term-risk states. This separation addresses the tension between retaining benign long-tail contributions and restricting harmful updates from clients with favorable histories. Execution and

current-update evidence inform the states, while persistence rules distinguish recoverable restrictions from permanent exclusion. In the reported mixed-attack experiments, this separation keeps permanent-ban FPR near zero while retaining high recall against the tested mixed attacks. The results relate the participation policy to both attack suppression and benign-client retention.

The evaluated profile is server-assisted vision FL with a

small trusted reference partition and an attestation-capable client path. The peer-reference branch provides a guiding direction when a clean validation gradient is unavailable. Microarchitectural side channels and confidential-VM resource contention remain outside the threat model. The probes are tailored to vision classification; report size and server audit overhead are kept separate from end-to-end latency. Cryptographic secure aggregation is outside the main boundary because the server must inspect individual evidence. This profile states where the trust-flow contract applies and provides a basis for extending it to other data modalities and deployment conditions.

References

- [1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguey y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent>
- [4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 5650–5659. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html>
- [5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in Sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/fung>
- [7] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, “FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2545–2555.
- [8] P. Rieger, T. D. Nguyen, M. Miettinen, and A. R. Sadeghi, “DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, “FLAME: Taming backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 1415–1432. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
- [10] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A. R. Sadeghi, “CrowdGuard: Federated backdoor detection in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [11] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, “RoseAgg: Robust defense against targeted collusion attacks in federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2951–2966, 2024.
- [12] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, “A training-integrity privacy-preserving federated learning scheme with trusted execution environment,” *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [13] T. Xu, K. Zhu, A. Andrzejak, and L. Zhang, “Distributed learning in trusted execution environment: A case study of federated learning in SGX,” in *Proceedings of the 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2021, pp. 450–454.
- [14] S. Queyruet, V. Schiavoni, and P. Felber, “Mitigating adversarial attacks in federated learning with trusted execution environments,” in *Proceedings of the IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 626–637.
- [15] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, “TMT-FL: Enabling trustworthy model training of federated learning with malicious participants,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2723–2740, 2025.
- [16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How To Backdoor Federated Learning,” in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, vol. 108, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [17] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks,” in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 707–723.
- [18] J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, “FLPurifier: Backdoor defense in federated learning via decoupled contrastive training,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4752–4766, 2024.
- [19] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, “ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.
- [20] L. Wang, M. Huang, Z. Zhang, M. Li, J. Wang, and K. Gai, “RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4280–4295, 2025.
- [21] X. Zhang, Z. Chen, G. Chen, X. Feng, Q. Shen, and Z. Wu, “RPPFL: Robust and privacy-preserving federated learning via trusted execution environments,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.